Dead on Arrival: Characterizing and Protecting Against Dead-Entry TLB Misses in GPU Microarchitectures

Abstract

GPU workloads with large memory footprints frequently suffer from redundant L2 TLB misses in which a recently evicted translation is immediately re-walked at full page-walk cost. We characterize these dead-entry misses *across 24 GPU workloads, finding they account for up to 99% of L2 TLB misses in the most TLB-sensitive applications, yet their performance impact varies widely depending on memory access structure. Workloads where warps share the same virtual page suffer from burst amplification, where a single eviction stalls many warps simultaneously waiting for one translation to return. In contrast, workloads where each warp accesses a distinct set of pages face a capacity-overflow problem that no replacement policy can resolve, a distinction validated by huge page experiments. Building on this two-class taxonomy, we design* DEPOT *(Dead-Entry PrOTection), a 1 KB Bloom filter mechanism that prevents recently evicted translations from being displaced immediately upon reinstallation, delivering up to 72% IPC improvement on interference-driven workloads with zero overhead on others, and composing with the state-of-the-art TLB prefetching and compaction mechanism, for 2 to 7% additional gain.*

***Index Terms*—GPU address translation, TLB, MSHR, memory hierarchy**

1. Introduction

Address translation is a first-order performance concern in modern GPU workloads [57]. As applications move toward larger, irregular memory footprints such as sparse graph traversal, scientific stencils, and machine learning inference over large embedding tables, the GPU L2 TLB becomes a sustained bottleneck [23, 61]. Prior work has approached this problem almost exclusively from an access-pattern perspective, predicting which virtual pages will be needed next and prefetching their translations before the demand miss occurs [4, 10, 28, 48]. This framing implicitly treats each TLB miss as a cold event requiring a fresh page walk, but many TLB misses are not cold at all. A *dead-entry TLB miss* occurs when a virtual page whose translation is still valid is evicted from the TLB and then re-accessed before the working set rotates again. The page walk is not cold; it is redundant. The entry was alive, was discarded by LRU replacement, and is now being reconstructed at full page-walk cost. Measuring this phenomenon across 24 GPU workloads reveals that dead-entry misses are pervasive: in the most TLB-sensitive applications, over 99% of all L2 TLB misses carry an eviction-history signature.

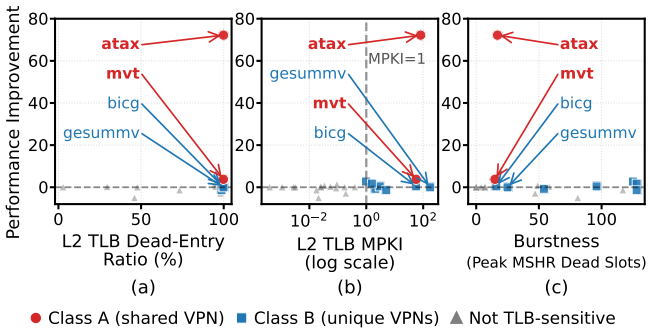

Figure 1: Performance improvement (IPC) from eliminating dead-entry re-walks against three candidate predictors: (a) DE ratio, (b) MPKI, and (c) MSHR burstiness, for all 24 workloads.

Yet the performance consequences of dead-entry misses are not uniform, and Figure 1 shows why simple miss statistics fail to explain the divergence. Each plot shows the performance improvement from eliminating dead-entry re-walks against a different candidate predictor for all 24 workloads. Figure 1(a) shows that dead-entry ratio has no predictive power: all TLB-sensitive workloads cluster near 99% dead-entry ratio, yet their performance improvement ranges from near zero to 72%. Figure 1(b) shows that MPKI is necessary but not sufficient: workloads with large IPC improvements and workloads with near-zero improvements remain indistinguishable at similar MPKI levels. Figure 1(c) reveals the discriminator: burstiness, measuring how many pending translation requests are collectively served by a single page walk; workloads with high burstiness achieve large performance improvements, while those with near-zero burstiness achieve near-zero improvements.

In some workloads, all threads in a warp reference the same memory page, so a single TLB eviction stalls many warps simultaneously waiting for one translation to return. Eliminating that redundant re-walk unblocks all of them at once, producing large performance improvements and high observed burstiness. In other workloads, each thread accesses a distinct page from a working set that exceeds TLB capacity. Evictions are driven by the sheer size of the working set, and no replacement-level mechanism can eliminate them because the TLB simply cannot hold all required translations at once. These two root causes lead to opposite outcomes under any

dead-entry protection mechanism, and we characterize and validate both across all 24 workloads in Section 5.

Building on this characterization, we design *DEPOT* (Dead-Entry PrOTection), a lightweight hardware mechanism that targets the first type of workload. DEPOT uses a small Bloom filter to track recently evicted TLB entries and temporarily protects re-installed entries from being displaced again, preventing the same redundant re-walks from compounding. The mechanism requires approximately 3.5 KB of storage, introduces no prediction logic, and falls back gracefully to standard LRU replacement for workloads where it provides no benefit. DEPOT also composes well with LatPC [28], the state-of-the-art TLB prefetching and compaction mechanism, because the two techniques address fundamentally different sources of misses. LatPC reduces misses through stride prediction and entry compaction; DEPOT prevents recently used pages from being discarded prematurely. Together they outperform either mechanism alone by 2 to 7% on TLB-sensitive workloads at negligible additional hardware cost.

In this paper, we make the following contributions.

- **Eviction-history characterization.** We measure GPU L2 TLB misses by eviction history across 24 workloads, establishing dead-entry re-walks as a prevalent, structurally distinct miss class.
- **Two-class taxonomy.** We identify two root causes among TLB-sensitive workloads, interference-driven misses where multiple warps share the same virtual page, and capacity-driven misses where the working set exceeds TLB reach, and validate the taxonomy using 2 MB huge page experiments.
- **DEPOT mechanism and composition.** We design DEPOT (Dead-Entry PrOTection), a Bloom-filter mechanism (1 KB filter, 3.5 KB total) achieving up to 72% performance improvement on interference-driven workloads and 2 to 7% additive improvement on top of LatPC [28], the state-of-the-art TLB prefetching and compaction mechanism.

The remainder of this paper is organized as follows. Section 2 reviews GPU address translation and dead-entry TLB misses. Section 3 summarizes related work. Section 4 describes the methodology and experimental setup. Section 5 presents the dead-entry miss characterization and workload taxonomy. Section 6 introduces DEPOT, and Section 7 evaluates its performance and composition with the state-of-the-art scheme. Section 8 discusses limitations, and Section 9 concludes.

2. Background

2.1. GPU Address Translation

Modern GPUs execute code in *warps* of 32 lock-step threads [1, 20, 24], so a single memory instruction can generate up to 32 virtual addresses at once. Coalescing [51] reduces this when threads share a page, but memory-intensive workloads with large or strided access patterns routinely generate one or more unique VPNs per warp instruction, creating translation pressure with no direct CPU analog.

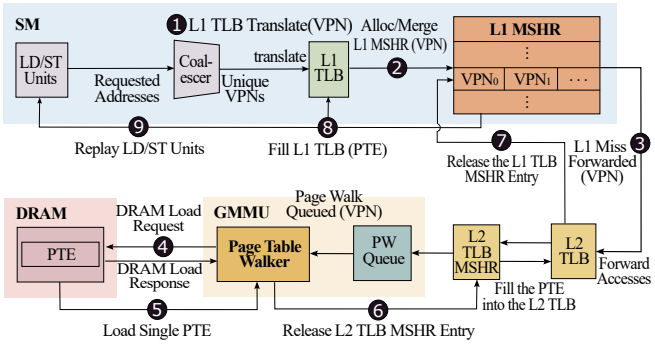

Figure 2: GPU address translation hierarchy (SM86): L1 TLB per SM, shared L2 TLB, 16 PTWs, and MSHR merging for same-VPN requests.

Figure 2 shows the pipeline. Each SM hosts a private, fully-associative 32-entry L1 TLB [56, 58], flushed at kernel boundaries; on the Ampere SM86 microarchitecture [50] studied here, 46 SMs operate independently. An L1 miss allocates an MSHR entry; a second request for the same in-flight VPN merges into it rather than issuing a duplicate walk, and both requestors unblock together on fill. Unmerged L1 misses forward to the shared, chip-wide L2 TLB (1024 entries on SM86, servicing all 46 SMs), which performs the same same-VPN merge before dispatching to one of 16 hardware page-table walkers (PTWs) [61], each traversing the page table via sequential DRAM reads at hundreds to over a thousand cycles. With 4 KB pages, 1024 L2 entries give 4 MB of reach; footprints beyond that miss regardless of replacement policy. Because only 16 PTWs exist, arriving misses queue behind occupied walkers, and because the L2 MSHR merges by VPN, a single eviction that many warps depend on stalls all of them at once when re-walked – the mechanism behind both the disproportionate cost of a dead-entry miss and the disproportionate recovery from protecting one. This PTW bottleneck is the primary reason TLB behavior dominates performance in translation-sensitive GPU workloads [5, 28, 56].

2.2. Dead-Entry TLB Misses

We define a *dead-entry TLB miss* formally:

> *A TLB miss at virtual page p at time t is a **dead-entry miss** if p was previously installed in the TLB, subsequently evicted before time t, and the eviction preceded the miss without any intervening access to p that would have re-established a valid entry.*

Equivalently, the page-table walker reconstructs a translation that was valid and present within the current execution window, then discarded by LRU replacement [11, 46] – the walk is logically redundant, not new or stale. The classical three-C taxonomy [29] describes *why* an eviction occurred; dead-entry is orthogonal, describing *what* the evicted entry was, and can arise from either a capacity eviction (working set exceeds reach) or an interference-driven eviction (a burst

of other pages displaces a still-needed entry) – the two root causes behind the workload classes of Section 5. Not every capacity or conflict eviction produces a dead-entry event: a page never re-accessed within the execution window produces no re-walk. This orthogonality is what makes dead-entry misses actionable where cold and capacity misses are not: a cold miss needs prediction, a capacity miss needs more reach, but a dead-entry miss is avoidable in principle simply by remembering the recent eviction and protecting the re-installed entry from immediate re-eviction – the observation that motivates DEPOT.

3. Related Work

TLB prefetching and compaction. LatPC [28] pairs a stride predictor [34] with entry compaction to prefetch ahead of demand misses; Valkyrie [10] extends this with multi-level, cross-SM prefetching, Avatar [53] across address-space boundaries, SnakeByte [40] via adaptive recursive page-merging, and STAR [41, 43] via sub-entry sharing on multi-instance GPUs. On CPUs, translation-triggered [12, 13] and learned/pipelined [44, 45] prefetching operate similarly. All predict future translations from access history; we instead characterize misses by eviction history, an orthogonal signal.
Page-walk optimization. R2D2 [27] exploits per-warp address linearity to eliminate redundant walks; Heliostat [21] repurposes ray-tracing hardware for parallel walks; Trans-FW [42] and Barre Chord [22] target multi-GPU/multi-chip-module translation; SpecTLB [7, 8] speculates ahead of confirmed translations and Griffin [9] supports multi-GPU page migration. These reduce the *cost* of walks that occur; we identify a complementary class where the walk itself is redundant.
Huge pages and page-table layout. Mosaic [4] provides range-TLB reach without OS support; CoLT [55] uses contiguous entries at 4 KB granularity; Elastic Cuckoo Page Tables [62] and Translation Ranger [35, 65] restructure layout for parallelism and contiguity-awareness; Pham et al. [52, 54] exploit natural translation clustering. These address the capacity limit directly; we use 2 MB pages only as a diagnostic to separate capacity-driven from interference-driven misses.
TLB-aware scheduling and dead-block prediction. CTA scheduling [39] and MASK [5, 56] improve locality and multi-application concurrency; OASIS [64] places memory object-aware across GPUs; batch-aware management [38, 61] amortizes translation overhead for irregular workloads – these reduce miss *frequency* through scheduling, whereas our structural cause persists regardless. Dead-block prediction has a long cache lineage [31–33]; Khan et al. [37] introduced sampling-based prediction for LLCs, and closest to our work, Mazumdar et al. [47] jointly predict dead pages and blocks via PC-based reuse. We instead use eviction history, requiring no PC tracking, and quantify prevalence across 24 GPU workloads – a structurally distinct miss class not previously isolated at the GPU L2 TLB.
MSHR and CPU TLB characterization. MSHR occupancy is known to carry workload-classification signal for cache/memory tuning [26, 59]; we apply an analogous signal to the TLB MSHR. CPU TLB studies [14, 15, 58] attribute misses mainly to working-set overflow; GPUs differ in that thousands of concurrent warps interact through one shared L2 TLB and MSHR, producing amplification effects absent on CPUs.

4. Methodology

4.1. Simulation Platform

All experiments run on Accel-Sim [36], a cycle-accurate GPU simulator built on GPGPU-Sim [6]. We adopt Accel-Sim's default `SM86_RTX3070` configuration [36], which models an NVIDIA GeForce RTX 3070-class GPU [49] based on the Ampere microarchitecture [2, 50]; the baseline GPU, cache, and memory parameters are used unmodified. The TLB hierarchy parameters—L1 and L2 TLB capacities, MSHR depths, lookup latencies, and the 16 page-table walkers—follow the configuration used in the state-of-the-art work [28], ensuring our results are directly comparable to that prior work. Table 1 summarizes the configuration. The L2 TLB has 1024 entries with LRU replacement, providing 4 MB of reach with 4 KB pages. Page-table walks are modeled with 16 dedicated PTWs, each capable of one concurrent walk. The L2 MSHR supports 128 in-flight translation requests; same-VPN requests are merged into a single MSHR entry.

TABLE 1 Simulated System Configuration.

GPU Configuration	
SM	46 SMs, 1536 threads / 32-thread warps, 32 CTAs per SM 65,536 registers per SM, up to 100 KB shared memory 1132 MHz core clock
Cache	128 B line; L1 D-cache 128 KB, 32-way, 384 MSHRs L2 cache (unified) 4 MB, 16-way
DRAM	GDDR6, 16 channels, 14 Gbps, 3500.5 MHz 254-cycle access latency
Virtual Memory Configuration	
L1 TLB	32 entries, fully associative, 4 KB pages 16 MSHRs (4-way merge), 4 ports 20-cycle lookup latency, LRU replacement
L2 TLB	1024 entries, 16-way, 4 KB pages 128 MSHRs (8-way merge), 16 ports 80-cycle lookup latency, LRU replacement Reach 4 MB (4 KB) / 256 MB (2 MB pages)
Page Walk	16 page-table walkers x86-64 4-level page table [3, 30], 254 cycles per level 32-entry page-walk cache, 20-cycle lookup

4.2. Instrumentation

We extend Accel-Sim with four measurement and mechanism modules:
Miss arrival timeseries: Records arrival timestamps of every L2 TLB miss, enabling analysis of temporal miss structure.
Dead-entry detection: Tracks the last eviction cycle per VPN in a hash table. A miss is classified as dead-entry if the VPN appears in this table, indicating a prior eviction since last installation.
MSHR dead-slot timeseries: Samples the number of MSHR entries currently waiting on dead-entry re-walks, at 100-cycle intervals. Peak occupancy defines the *burstiness* metric.

TABLE 2 List of Evaluated Workloads.

Workload	Suite	MPKI	mem-MPKI	Footprint	Cls
atax	PolyBench	81.4	119.6	16.0 MB	A
mvt	PolyBench	56.5	83.0	16.0 MB	A
bicg	PolyBench	56.4	82.8	16.0 MB	B
gesummv	PolyBench	175.5	249.7	31.3 MB	B
nw	Rodinia	5.02	67.9	32.0 MB	B
dmr	Lonestar	2.99	18.6	24.2 MB	B
kmeans	Rodinia	2.09	72.5	30.3 MB	B
sssp	Lonestar	1.58	11.3	48.0 MB	B
mst	Lonestar	0.98	6.80	73.0 MB	B
backprop	Rodinia	0.02	0.30	9.0 MB	—
bfs	Rodinia	0.03	0.20	2.4 MB	—
gaussian	Rodinia	0.00	0.01	0.5 MB	—
hybridsort	Rodinia	0.02	0.46	12.0 MB	—
lud	Rodinia	0.03	0.58	16.0 MB	—
pagerank	Pannotia	0.06	0.40	10.9 MB	—
p-bfs	Parboil	0.00	0.07	1.1 MB	—
mri-gridding	Parboil	0.09	6.04	226.4 MB	—
sad	Parboil	0.02	1.48	8.5 MB	—
sgemm	Parboil	0.00	0.07	12.0 MB	—
spmv	Parboil	0.19	0.71	29.4 MB	—
2mm	PolyBench	0.00	0.00	5.0 MB	—
fdtd2d	PolyBench	0.09	0.49	48.0 MB	—
gramschmidt	PolyBench	0.40	0.62	24.4 MB	—
streamcluster	Rodinia	0.17	0.70	8.4 MB	—

DEPOT protection statistics: Records Bloom filter insert and hit counts, protected-entry counts, and LRU fallback events. More implementation details are in Section 6.

4.3. Workload Suite

Table 2 characterizes all 24 workloads (Classes: **A** = interference-driven; **B** = capacity-driven; **—** = not TLB-sensitive). Here, MPKI is L2 TLB misses per kilo-instruction, while mem-MPKI is misses per kilo memory-instruction, isolating translation pressure from non-memory compute. We select applications with working sets exceeding 4 MB or L2 TLB MPKI above 1.0, matching the criteria used in LatPC [28]. Workloads span Rodinia [19], PolyBench [25], Lonestar [17], Parboil [63], and Pannotia [18] benchmark suites. We apply a two-bin framing: **TLB-sensitive** workloads (L2 MPKI $\geq$ 1, matching the selection criterion of LatPC [28]) vs. **not TLB-sensitive** workloads (L2 MPKI $<$ 1). Nine workloads meet the TLB-sensitive threshold (including `mst` at MPKI = 0.98, a near-threshold borderline case with confirmed capacity-driven root cause); the remaining 15 serve as a graceful-degradation validation set. Within the TLB-sensitive bin, we sub-classify by access structure: **Class A** (interference-driven) for workloads where all threads in a warp reference the same virtual page (shared VPN)—producing MSHR-merge amplification—and **Class B** (capacity-driven) for workloads where each warp instruction generates distinct VPNs per thread, driving working-set-overflow evictions that exceed TLB reach. Class B membership is confirmed by the 2 MB IPC experiment: if switching to 2 MB pages (L2 reach = 256 MB) yields a large IPC speedup, the workload was reach-limited.

4.4. Evaluated Configurations

We evaluate six configurations: (i) Baseline: Unmodified LRU replacement, 4 KB pages, (ii) DEPOT: DEPOT eviction protection enabled with default parameters (W = 500K cycles, 8192-bit Bloom filter), (iii) 2 MB baseline: Baseline with 2 MB huge pages (L2 TLB reconfigured to 128 entries $\times$ 2 MB, reach = 256 MB), (iv) 2 MB+DEPOT: DEPOT with 2 MB pages, (v) LatPC: State-of-the-art TLB prefetching and compaction [28] enabled, 4 KB pages, and (vi) LatPC+DEPOT: Both LatPC and DEPOT enabled simultaneously.

5. Dead-Entry Miss Characterization

5.1. Dead-Entry Ratio

Figure 3 shows the dead-entry ratio, defined as the fraction of L2 TLB misses classified as dead-entry re-walks, for all 24 workloads sorted in descending order. The distribution is striking: dead-entry misses are not a rare edge case but the dominant miss type across the benchmark suite. 16 of the 24 workloads exhibit dead-entry ratios above 90%, and all 9 TLB-sensitive workloads exceed 98% (six of them above 99%). This near-unity ratio arises from the interplay between working-set size and access recurrence. Once a TLB-sensitive workload reaches steady-state execution, its working set is large enough to guarantee eviction between successive accesses to the same page, yet small enough that the same pages are accessed repeatedly within a single kernel invocation. As a result, virtually every L2 TLB miss encountered in steady state is a re-walk of a page that the TLB has already translated and discarded.

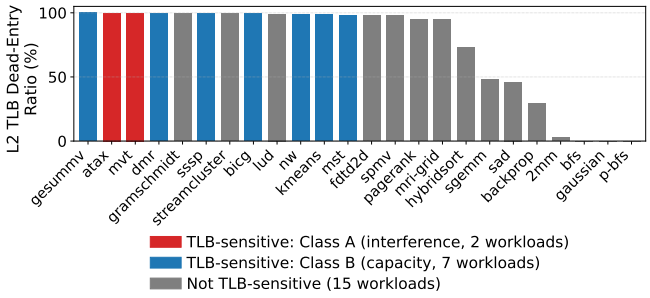

Figure 3: L2 TLB dead-entry ratio for all 24 workloads, sorted descending.

Importantly, high DE ratio alone does not imply TLB sensitivity. Workloads like `lud` and `gramschmidt` exceed 99% DE ratio yet sustain high throughput because their absolute miss rate is low enough that cumulative stall time remains negligible, motivating a joint condition of high DE ratio and high MPKI.

Figure 4 plots all 24 workloads by DE ratio against L2 TLB MPKI. The most immediately apparent feature is the vertical gap separating TLB-sensitive workloads from the rest: 9 sensitive workloads occupy a clearly elevated MPKI band, while the remaining 15 cluster near the bottom of the MPKI axis regardless of their DE ratio. Within the

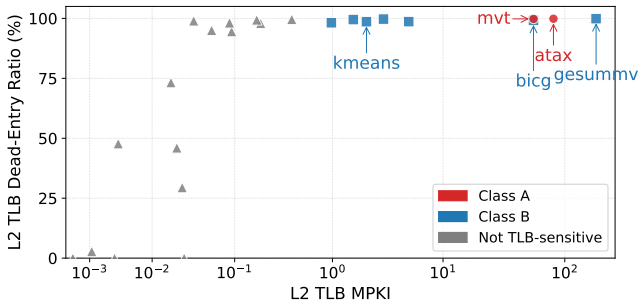

Figure 4: L2 TLB Dead-entry ratio vs. L2 TLB MPKI for all 24 workloads.

high-MPKI band, all workloads cluster near 100% dead-entry ratio, confirming that near-unity DE ratio is a reliable characteristic of workloads that are both TLB-sensitive and operating in steady state. In contrast, the lower portion of the plot shows a wide spread of DE ratios among workloads with low MPKI, ranging from near zero to above 99%, confirming that DE ratio alone carries no predictive power for performance sensitivity.

5.2. Temporal Structure

To understand how dead-entry misses evolve over the course of kernel execution, Figure 5 shows the miss arrival timeseries for `atax` and `bicg`, two workloads that share nearly identical DE ratios (∼99.9% and ∼99.2%) and comparable MPKI (81 and 56, respectively) yet respond very differently to eviction-history protection.

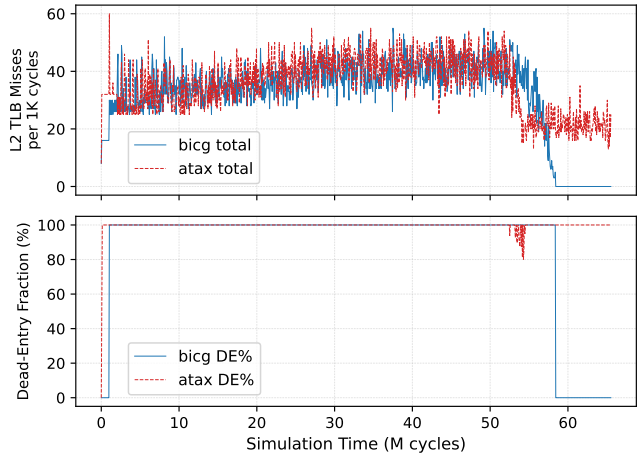

Figure 5: Miss arrival timeseries and dead-entry fraction for `atax` and `bicg` in steady state.

In both workloads, the dead-entry fraction rises sharply from cold-start and stabilizes near 99% within the first few thousand kernel cycles. This rapid convergence shows that the dead-entry phenomenon is a steady-state property of the workload's memory access pattern rather than a transient warm-up artifact. After the initial cold-miss period, the TLB is fully populated with the working set and subsequent misses are almost entirely re-walks of evicted entries. The miss arrival rate, shown as the total number of misses per time window, is comparable between the two workloads throughout steady state. Both issue misses at similar frequencies and both carry the eviction-history signature on nearly all of them. The timeseries data alone would lead to the conclusion that the two workloads are structurally interchangeable, yet their performance responses to the same eviction-history intervention differ by nearly two orders of magnitude. Understanding this divergence requires examining beyond miss rate to how warps share and reuse virtual pages.

5.3. Access Structure: The Class Discriminator

The discriminator between `atax` and `bicg` is not miss frequency but access structure, specifically whether warps tend to reference the same virtual page or distinct virtual pages at a given moment. Figure 6 shows the MSHR dead-slot occupancy timeseries for both workloads. `atax` exhibits brief, high-amplitude bursts separated by near-zero occupancy, while `bicg` shows a flat, sustained occupancy level throughout execution. This difference directly reflects how each workload maps onto the warp execution model.

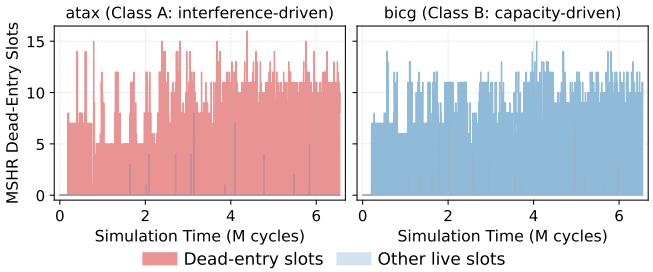

Figure 6: MSHR dead-slot occupancy over time for `atax` and `bicg`.

In `atax`, each warp computes y += $A[i][j] \cdot x[j]$. All 32 threads within the warp access the same element of the x[] vector, the same j index, producing a single shared VPN for that warp instruction. When this VPN is evicted and re-walked, all warps stalled on the same $x[j]$ reference are merged into a single MSHR entry through same-VPN coalescing. A single page walk therefore serves many waiting warps simultaneously, producing the sharp occupancy spikes in Figure 6. The spike height reflects the number of merged and simultaneously released warps. `mvt` exhibits the same access pattern for analogous algebraic reasons, making it the second member of this class.

In `bicg`, each warp computes a column dot product where thread i accesses row i of matrix A, generating 32 distinct VPNs per warp instruction with little cross-warp sharing. The MSHR merge depth is therefore approximately one, so each miss requires an independent page walk. Its 2048-by-2048 working set requires approximately 4096 pages, four times the 1024-entry L2 TLB capacity, making evictions fundamentally capacity-driven. Consequently, the MSHR occupancy trace in Figure 6 remains flat and sustained throughout execution because misses form a steady stream

of independent capacity evictions rather than burst events. The remaining 6 Class B workloads (`nw`, `sssp`, `kmeans`, `gesummv`, `dmr`, and `mst`) exhibit the same structural property, generating unique VPN streams with working sets that exceed L2 TLB reach.

The two classes can therefore be distinguished by the shape of the MSHR dead-slot occupancy signal. Bursty, high-amplitude spikes indicate that warps share virtual pages and that individual eviction events serve many waiting requestors at once. Flat, sustained occupancy indicates that each warp accesses distinct pages and that evictions are driven by working-set capacity that cannot be resolved at the replacement level. This temporal signature is observable from existing MSHR occupancy counters without any offline profiling or application modification, making it a practical runtime discriminator.

5.4. Validating Capacity-Driven Misses

The classification of seven workloads as capacity-driven rests on the claim that their dead-entry misses originate from working-set overflow rather than from interference at the TLB replacement level. A direct way to test this claim is to expand TLB reach and observe whether the misses disappear. Figure 7 shows IPC speedup under 2 MB huge pages relative to the 4 KB baseline for all nine TLB-sensitive workloads. With 2 MB pages, the L2 TLB reach expands from 4 MB to 256 MB, covering a far larger fraction of each workload's active footprint with the same 1024 entries.

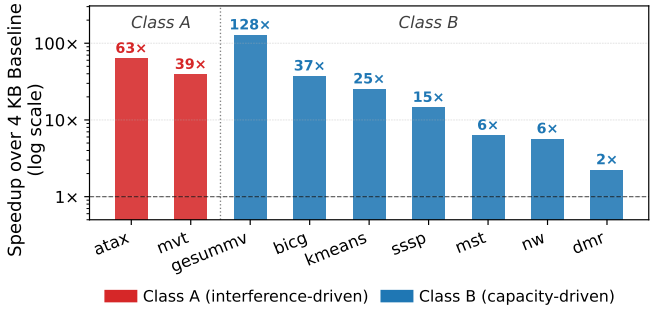

Figure 7: IPC speedup under 2 MB pages over 4 KB baseline for all 9 TLB-sensitive workloads.

The results validate the capacity-driven diagnosis across all 7 Class B workloads through a wide range of speedups that reflect each workload's working-set size relative to the expanded reach. `gesummv` shows the most dramatic response at 128×: its 16 MB working set fits entirely within the 256 MB reach, eliminating capacity evictions almost completely. `kmeans` (25×), `sssp` (14.5×), `bicg` (37×), `nw` (5.7×), and `mst` (6.3×) each improve by an amount that tracks how tightly their active footprint is bounded by the reach ceiling. `dmr` shows the smallest response at 2.2×, reflecting a more irregular address stream only partially contained within the larger reach. In every case the improvement confirms that capacity overflow is the root cause. The two Class A workloads, `atax` and `mvt`, also benefit substantially from 2 MB pages, achieving 63× and 39× speedups, respectively. However, unlike the Class B workloads, their performance is not fundamentally constrained by working-set capacity: expanding reach does not eliminate the interference that causes evictions when many warps compete for the same entries.

6. DEPOT: Dead-Entry PrOTection

In this section, we propose DEPOT that targets Class A dead-entry re-walks through a three-phase mechanism detection, fill, and eviction implemented entirely within the L2 TLB pipeline. Figure 8 shows the hardware. Three additions extend the existing L2 TLB datapath: an eviction-history Bloom filter, a small Pending Dead-Entry Set (PDS) that bridges the miss and fill events, and a Protection Timestamp Writer that drives a new 20-bit per-entry protection-timer field in the tag array.

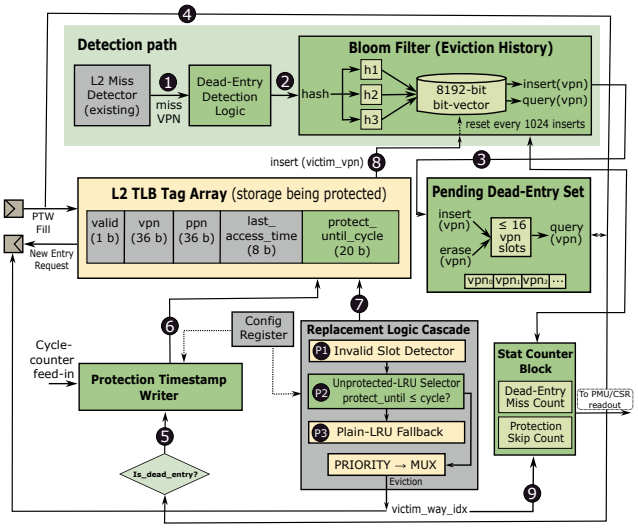

Figure 8: DEPOT mechanism: eviction-history Bloom filter, Pending Dead-Entry Set, per-entry protection timer, and protection-aware replacement cascade.

Detection (miss path). On every L2 TLB miss, the Dead-Entry Detection Logic receives the missed VPN ❶ and hashes it through three independent hash functions (h_1, h_2, h_3) into an 8192-bit Bloom filter [16] that records recently evicted VPNs ❷. A filter hit indicates that the missed VPN was recently in the TLB and is therefore a probable dead-entry re-walk, so the in-flight miss is registered in the PDS ❸-a small ($\leq$ 16 slot) buffer that holds VPNs awaiting fill. Misses whose VPN is absent from the filter (genuine cold or capacity misses with no eviction history) bypass the PDS and proceed through the standard page-walk path.

Fill (protection path). When the page-table walk completes, the fill triggers a PDS query for the filling VPN ❹. A PDS hit activates the Protection Timestamp Writer ❺, which reads the cycle counter and sets the newly installed entry's protection timer to expire at $now + W$ cycles ❻, where W defaults to 500 K cycles and is held in a configuration register. The PDS

slot for that VPN is then released. Fills whose VPN was not previously flagged install with the protection field untouched and behave identically to baseline.

Eviction (protection-aware replacement). On every fill that requires a victim, the Replacement Logic Cascade reads each entry's protection timer from the tag array ❼ and selects through three priority stages: (P1) *Invalid Slot Detector* returns any invalid entry immediately; (P2) *Unprotected-LRU Selector* compares each entry's protection timer against the current cycle and selects the LRU entry among those whose protection has expired; (P3) *Plain-LRU Fallback* fires only when every candidate in the set is still protected, in which case the cascade reverts to standard LRU. The selected victim's VPN is inserted into the Bloom filter, refreshing the eviction history ❽, and the victim's way index is forwarded to a small Stat Counter Block ❾ that records dead-entry-miss and protection-skip counts. This graceful degradation is a correctness property: it bounds the worst-case overhead of DEPOT to zero, since the fallback path is indistinguishable from baseline LRU. Two operational details keep the mechanism stable across long-running workloads. First, the filter is reset after every 1024 insertions rather than on a fixed time interval; this count-based threshold keeps the expected false-positive rate below 5% while avoiding the temporal aliasing of a cycle-timer reset that could misfire across kernel boundaries. Second, a protection-state flush at each kernel boundary clears all protection timers, ensuring that a VPN evicted during kernel K_n does not spuriously protect an entry in kernel K_{n+1}, where the same virtual address may map to a different physical frame after relaunch.

Regarding hardware cost, the Bloom filter occupies 1 KB of storage. Each TLB entry requires one additional per-entry protection timer of 20 bits to record the expiry cycle, yielding $1024 \times 20 = 20$ Kbits = 2.5 KB of per-entry SRAM across the full 1024-entry L2 TLB. The total overhead is therefore 1 KB + 2.5 KB = 3.5 KB, amortized across the existing TLB array. DEPOT's eviction-history Bloom filter is structurally similar to the evicted-address filter introduced by Seshadri et al. [60] for cache pollution and thrashing mitigation; we apply the same primitive at the TLB layer, where eviction-history information drives protection rather than insertion policy.

7. Performance Evaluation and Analysis

7.1. DEPOT Performance

Figure 9 shows IPC improvement of DEPOT for all 24 workloads. The results divide cleanly along the class boundaries established in Section 5. Among the 15 not-TLB-sensitive workloads, IPC improvement is near zero throughout, with occasional small negative values including `sad` at –5.1% and `spmv` at –2.9%. These minor regressions arise because DEPOT occasionally fires on workloads with moderate DE ratios, marking re-installed entries as protected even though TLB misses are not on their critical path; with MPKI of 0.02 and 0.19 respectively, `sad` and `spmv` incur negligible TLB stall time at baseline, so the observed losses reflect second-order LRU displacement rather than

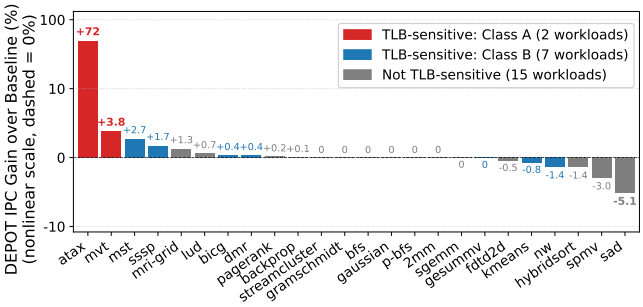

Figure 9: IPC improvement of DEPOT over 4 KB baseline for all 24 workloads, sorted descending.

translation overhead. The LRU fallback bounds worst-case degradation but activates only when all entries in a set are simultaneously protected, so partial-protection sets can still make suboptimal eviction choices, which accounts for the small residual regression. Among the two Class A workloads, `atax` improves from 0.354 to 0.610 IPC for a gain of +72%, reflecting near-complete elimination of dead-entry re-walk stalls that previously dominated its execution time. `mvt` improves from 0.574 to 0.596 IPC for a gain of +3.8%. The smaller gain on `mvt` reflects its peak MSHR merge depth of 15 versus `atax`'s 17 and its higher baseline IPC, indicating that TLB stalls already represent a smaller fraction of its execution time before DEPOT is applied. Both workloads converge to similar post-DEPOT IPC values near 0.60, consistent with reaching a shared performance ceiling determined by compute throughput and memory bandwidth rather than translation overhead. All seven Class B workloads show near-zero DEPOT gain, ranging from –1.4% to +2.7%, within simulation noise. This result holds across PolyBench, Lonestar, and Rodinia workloads: when each warp generates distinct VPNs, protecting one re-installed entry is immediately offset by another capacity eviction, while the LRU fallback ensures zero overhead.

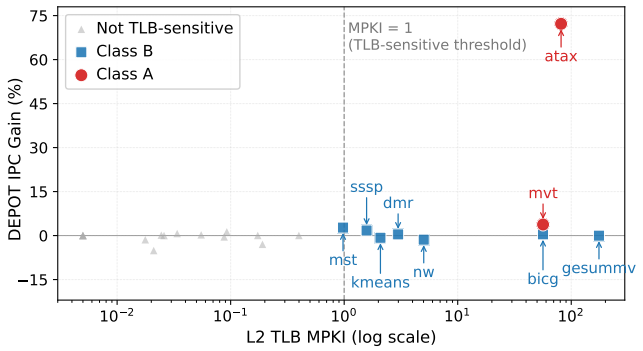

Figure 10: IPC improvement of DEPOT vs. L2 TLB MPKI for all 24 workloads.

Figure 10 plots IPC improvement of DEPOT against MPKI for all 24 workloads, revealing a clean two-regime structure. Workloads with MPKI below 1 cluster near zero gain regardless of their dead-entry ratio, confirming that TLB

miss rate is a necessary condition for sensitivity. Within the high-MPKI group, workloads split by access structure: those with shared VPN access show positive gains, while those with unique VPN access per warp show near-zero gains. The scatter plot makes visible what the aggregate bar chart in Figure 9 obscures: MPKI identifies workloads for which TLB behavior matters at all, while access structure determines whether eviction-history protection can improve it. To bound the overhead from false positives, we also evaluate a worst-case scenario with a fully saturated Bloom filter, where every miss query returns a positive response regardless of actual eviction history. Under saturation, `atax` IPC remains at 0.610, identical to the normal DEPOT result. False positives cause DEPOT to protect entries that did not originate from dead-entry re-walks, but the LRU fallback absorbs the resulting mismatch without measurable performance impact, showing that the mechanism is tolerant of false positive pressure.

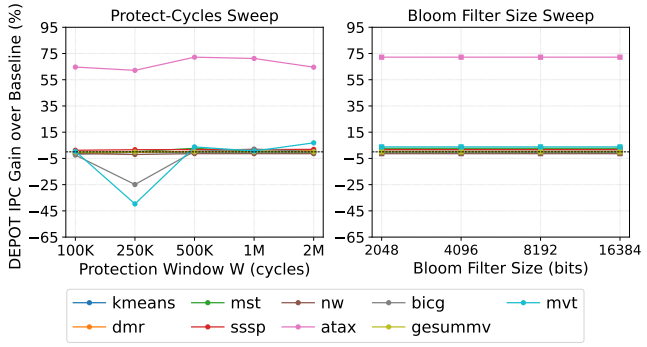

Figure 11: DEPOT parameter sensitivity for 9 TLB-sensitive workloads: protection window W (left) and Bloom filter size (right).

Figure 11 shows DEPOT sensitivity to its two configurable parameters for 9 TLB-sensitive workloads. Both the protection-window sweep (100 K to 2 M) and Bloom-filter sweep (2048 to 16384 bits) remain nearly flat across the tested range. This indicates that TLB-sensitive workloads experience re-walk bursts within timescales covered by any reasonable protection window, while the eviction stream remains well below the saturation threshold of even the smallest Bloom filter. DEPOT is therefore robust to parameter choice, and the default configuration of a 500 K-cycle protection window and an 8192-bit Bloom filter is effective across all evaluated workloads without tuning.

7.2. Class A/B Asymmetry

The performance asymmetry between Class A and Class B follows directly from the access-structure analysis in Section 5.3 and explains both the large gains on `atax` and the near-zero results on the Class B workloads. In Class A workloads, a single dead-entry eviction is amplified through MSHR merging because many warps simultaneously reference the same virtual page. When DEPOT protects the re-installed entry, it eliminates both the initial re-walk and the merged re-walks that would otherwise stall many warps simultaneously. On SM86, page-walk latency is approximately 300–500 cycles, and releasing dozens of stalled warps with a single protection event can recover thousands of cycles, consistent with the observed +72% IPC improvement.

In Class B workloads, by contrast, the TLB is continuously occupied by a working set that exceeds TLB reach, and each warp accesses distinct pages with little cross-warp sharing. Protecting one re-installed entry therefore only shifts the next eviction to another entry without reducing the overall eviction rate. Because MSHR merging is minimal, each page walk serves only one warp, so the benefit of protection is limited. When all entries are under protection pressure, DEPOT falls back to standard LRU replacement, ensuring negligible overhead in this regime.

7.3. Composition with LatPC

LatPC combines stride-based prefetching with entry compaction and represents the state-of-the-art in GPU TLB optimization. It trains a stride predictor for each TLB entry and issues prefetch requests before demand misses occur, targeting cold and capacity-driven misses through access-pattern prediction. DEPOT and LatPC address fundamentally different sources of TLB misses: LatPC reduces misses by predicting future accesses and filling the TLB ahead of demand, while DEPOT reduces misses by preventing recently installed entries from being prematurely evicted at the replacement boundary. Because their mechanisms operate on different points in the TLB pipeline, one does not subsume the other: on interference-driven workloads the two compose additively, while on capacity-saturated workloads they can contend, as the results below show.

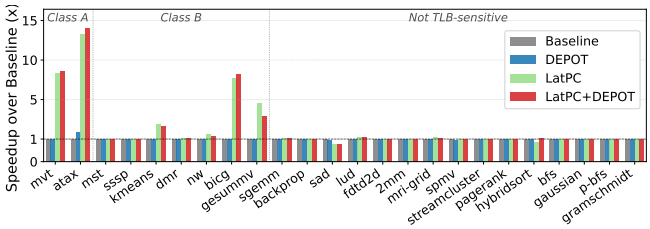

Figure 12: IPC comparison across four configurations for all 24 workloads.

Figure 12 shows the composition result across all configurations. LatPC alone achieves large gains on TLB-sensitive workloads, with `atax` improving by +1227% from 0.354 to 4.70 IPC, `bicg` by +667% from 0.601 to 4.61 IPC, and `mvt` by +729% from 0.574 to 4.76 IPC. These gains reflect LatPC's elimination of the dominant cold-miss bottleneck, which accounts for the large majority of TLB stalls in these workloads. LatPC+DEPOT outperforms LatPC alone on the Class A workloads, with `atax` improving further from 4.70 to 4.95 IPC (+5.4%) and `mvt` from 4.76 to 4.95 IPC (+4.0%). Across the Class B workloads, LatPC+DEPOT shows mixed results: `bicg` (+6.9%), `mst` (+2.5%), and `sssp` (+2.1%) gain modestly, `dmr` is negligible (–0.9%), while `kmeans` (–8.3%), `nw` (–10.0%), and `gesummv` (–28.5%) regress.

These regressions arise because DEPOT's eviction-protection policy conflicts with LatPC's prefetch-driven replacement in capacity-saturated workloads: LatPC must freely evict entries to install ahead-of-demand translations, and DEPOT's protected entries block those slots, delaying prefetch fills and causing net IPC loss.

The additive improvement on Class A workloads arises because LatPC does not eliminate all dead-entry re-walks even when its prefetcher is operating correctly. The key reason is the asymmetry between what the prefetcher targets and what eviction produces. LatPC observes that a warp accesses VPN *A*, then predicts and prefetches the next VPN in the stride sequence. While the prefetch is in flight, VPN *A* itself may be displaced from the TLB by LRU replacement, because the prefetch installs a new entry and the LRU policy does not know that *A* will be accessed again at the next stride period. When that next access to *A* arrives it finds the entry gone, a dead-entry miss that LatPC's forward-looking predictor cannot catch; DEPOT observes the eviction of *A* at the replacement boundary and protects the re-installed entry, covering exactly this residual class. Where the combination is beneficial—the Class A workloads and `bicg`—the 2 to 7% additional IPC improvement reflects how frequently this interaction between prefetch-driven installation and LRU displacement produces residual dead-entry events. The marginal hardware cost of adding DEPOT to a LatPC-equipped TLB is a 1 KB Bloom filter, negligible relative to LatPC's per-entry stride prediction tables, making the combination an efficient pairing for workloads that benefit from both.

8. Discussion and Limitations

The 2 MB page results in Section 5.4 raise a natural question about whether DEPOT is necessary if larger pages already resolve TLB pressure. The two mechanisms address different root causes and are not interchangeable. Larger pages expand TLB reach and directly relieve capacity eviction in Class B workloads, but they do not eliminate interference-driven eviction in Class A workloads, nor are they always available: GPU workloads frequently use custom memory pools or third-party libraries that do not guarantee 2 MB alignment, and virtualized environments may not expose huge page support to guest workloads at all. DEPOT operates entirely within the TLB microarchitecture and is transparent to the OS, driver, and application, making it applicable in settings where large pages are unavailable or impractical.

DEPOT targets interference-driven dead-entry misses and provides no benefit for workloads whose TLB pressure arises from working-set overflow beyond the TLB reach. The Class B workloads in this study fall squarely into this regime, and the near-zero DEPOT gains on those workloads confirm that eviction-history protection does not substitute for additional TLB capacity or larger page mappings. Practitioners diagnosing TLB performance issues should therefore use the two-class characterization framework to identify the dominant root cause before selecting an intervention, as applying DEPOT to a capacity-driven workload not only fails to improve performance but can actively regress it when a prefetcher such as LatPC is also present, by blocking the replacement slots that prefetch fills require.

Composition with an aggressive prefetcher is a further limitation. DEPOT applied alone never regresses, but composing it with LatPC is not universally additive: on three capacity-driven Class B workloads—`kmeans`, `nw`, and `gesummv`—LatPC+DEPOT regresses relative to LatPC alone (by 8.3%, 10.0%, and 28.5% respectively), as DEPOT's protection window and LatPC's prefetch installs contend for L2 TLB capacity. This is consistent with the taxonomy rather than a counterexample to it: DEPOT is an interference-driven (Class A) optimization, and these regressions occur only when it is misapplied to capacity-driven workloads atop a prefetcher. A capacity-aware coupling of the two mechanisms—for instance, suspending protection while the prefetcher is actively installing—is a worthwhile direction for future work.

The evaluation in this paper is conducted on the SM86 Ampere microarchitecture using a validated simulation model. The qualitative behavior of dead-entry misses, including MSHR-merge amplification and the distinction between interference-driven and capacity-driven eviction, is expected to generalize across GPU microarchitectures that share the same L1 and L2 TLB organization. However, the specific quantitative thresholds used to classify workloads and the absolute IPC improvement reported here are calibrated to SM86 parameters such as the 1024-entry L2 TLB capacity and 16 hardware page-table walkers, and may differ on architectures with substantially different TLB configurations.

9. Conclusion

Dead-entry TLB misses are a structurally distinct, avoidable miss class that accounts for over 99% of L2 TLB misses in the most TLB-sensitive GPU workloads. We characterize dead-entry misses across 24 GPU workloads, using MPKI ≥ 1 as a TLB-sensitivity threshold to identify 9 TLB-sensitive workloads and sub-classify them by access structure into Class A and Class B, with Class B membership validated by 2 MB IPC experiments confirming reach-limited root causes. DEPOT, a 1 KB Bloom-filter eviction protection mechanism, eliminates Class A dead-entry re-walks with up to 72% IPC improvement, degrades gracefully to zero overhead on Class B, and composes additively with LatPC [28] on interference-driven workloads to deliver 2 to 7% further improvement at negligible marginal hardware cost, pushing `atax` from 4.70 to 4.95 IPC for a combined 14× improvement over the 4 KB baseline. Several directions remain for future work. The temporal MSHR occupancy signatures identified in this paper suggest the potential for runtime-adaptive TLB policies that dynamically distinguish interference-driven from capacity-driven miss behavior. In addition, integrating eviction-history signals with TLB-aware warp scheduling or page-placement mechanisms may further reduce destructive translation interference before evictions occur.

References

[1] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool, 2018.

[2] H. Abdelkhalik, Y. Arafa, N. Santhi, and A.-H. Badawy, "Demystifying the Nvidia Ampere architecture through microbenchmarking and instruction-level analysis," arXiv:2208.11174, 2022.

[3] Advanced Micro Devices, Inc., "AMD64 architecture programmer's manual," 2024.

[4] R. Ausavarungnirun, J. Landgraf, V. Miller, S. Ghose, J. Gandhi, C. J. Rossbach, and O. Mutlu, "Mosaic: A GPU memory manager with application-transparent support for multiple page sizes," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2017.

[5] R. Ausavarungnirun, V. Miller, J. Landgraf, S. Ghose, J. Gandhi, A. Jog, C. J. Rossbach, and O. Mutlu, "MASK: Redesigning the GPU memory hierarchy to support multi-application concurrency," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2018.

[6] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proc. International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009.

[7] T. W. Barr, A. L. Cox, and S. Rixner, "Translation caching: Skip, don't walk (the page table)," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2010.

[8] T. W. Barr, A. L. Cox, and S. Rixner, "SpecTLB: A mechanism for speculative address translation," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2011.

[9] T. Baruah, Y. Sun, A. T. Dincer, S. A. Mojumder, J. L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, and D. Kaeli, "Griffin: Hardware-software support for efficient page migration in multi-GPU systems," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.

[10] T. Baruah, Y. Sun, S. A. Mojumder, J. L. Abellán, Y. Ukidave, A. Joshi, N. Rubin, J. Kim, and D. Kaeli, "Valkyrie: Leveraging inter-TLB locality to enhance GPU performance," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2020.

[11] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[12] A. Bhattacharjee, "Large-reach memory management unit caches," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2013.

[13] A. Bhattacharjee, "Translation-triggered prefetching," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[14] A. Bhattacharjee, D. Lustig, and M. Martonosi, "Shared last-level TLBs for chip multiprocessors," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2011.

[15] A. Bhattacharjee and M. Martonosi, "Inter-core cooperative TLB prefetchers for chip multiprocessors," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2010.

[16] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[17] M. Burtscher, R. Nasre, and K. Pingali, "A quantitative study of irregular programs on GPUs," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2012.

[18] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding irregular GPGPU graph applications," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2013.

[19] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, 2009.

[20] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, and K. Skadron, "A performance study of general-purpose applications on graphics processors using CUDA," *Journal of Parallel and Distributed Computing*, vol. 68, no. 10, 2008.

[21] Y. Feng, Y. Li, J. Lee, W. W. Ro, and H. Jeon, "Heliostat: Harnessing ray tracing accelerators for page table walks," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.

[22] Y. Feng, S. Na, H. Kim, and H. Jeon, "Barre chord: Efficient virtual memory translation for multi-chip-module GPUs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.

[23] D. Ganguly, Z. Zhang, J. Yang, and R. Melhem, "Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2019.

[24] M. Garland, S. Le Grand, J. Nickolls, J. Anderson, J. Hardwick, S. Morton, E. Phillips, Y. Zhang, and V. Volkov, "Parallel computing experiences with CUDA," *IEEE Micro*, vol. 28, no. 4, 2008.

[25] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to GPU codes," in *Proc. Innovative Parallel Computing (InPar)*, 2012.

[26] Y. Gu and L. Chen, "Dynamically linked MSHRs for adaptive miss handling in GPUs," in *Proc. International Conference on Supercomputing (ICS)*, 2019.

[27] D. Ha, Y. Oh, and W. W. Ro, "R2D2: Removing redundancy utilizing linearity of address generation in GPUs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2023.

[28] Y. Ha *et al.*, "LATPC: Locality-aware TLB prefetching and MSHR compression," in *Proc. International Symposium on Microarchitecture (MICRO)*, Seoul, Republic of Korea, Oct. 2025.

[29] M. D. Hill and A. J. Smith, "Evaluating associativity in CPU caches," *IEEE Transactions on Computers*, vol. 38, no. 12, pp. 1612–1630, 1989.

[30] Intel Corporation, "Intel® 64 and IA-32 architectures software developer's manual," 2024.

[31] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2016.

[32] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2010.

[33] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proc. International Symposium on Computer Architecture (ISCA)*, 1990.

[34] G. B. Kandiraju and A. Sivasubramaniam, "Going the distance for TLB prefetching: An application-driven study," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2002.

[35] V. Karakostas, J. Gandhi, F. Ayar, A. Cristal, M. D. Hill, K. S. McKinley, M. Nemirovsky, M. M. Swift, and O. Ünsal, "Redundant memory mappings for fast access to large memories," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2015.

[36] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2020.

[37] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2010.

[38] H. Kim, J. Sim, P. Gera, R. Hadidi, and H. Kim, "Batch-aware unified memory management in GPUs for irregular workloads," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[39] G. Koo, H. Jeon, Z. Liu, N. S. Sung, and M. Annavaram, "CTA-aware prefetching and scheduling for GPU," in *Proc. IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018.

[40] J. Lee, J. M. Lee, Y. Oh, W. J. Song, and W. W. Ro, "SnakeByte: A TLB design with adaptive and recursive page merging in GPUs," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[41] B. Li, Y. Wang, T. Wang, L. Eeckhout, J. Yang, and X. Tang, "STAR: Sub-entry sharing-aware TLB for multi-instance GPU," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2024.

[42] B. Li, J. Yin, A. Holey, Y. Zhang, J. Yang, and X. Tang, "Trans-FW: Short circuiting page table walk in multi-GPU systems via remote forwarding," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.

[43] B. Li, J. Yin, Y. Zhang, and X. Tang, "Improving address translation in multi-GPUs via sharing and spilling aware TLB design," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2021.

[44] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Virtual address translation via learned page table indexes," in *Workshop on ML for Systems at NeurIPS*, 2018.

[45] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Prefetched address translation," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2019.

[46] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[47] C. Mazumdar, P. Mitra, and A. Basu, "Dead page and dead block predictors: Cleaning TLBs and caches together," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2021.

[48] S. Mostofi, H. Falahati, N. Mahani, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Snake: A variable-length chain-based prefetching for GPUs," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2023.

[49] NVIDIA Corporation, "NVIDIA GeForce RTX 3070 family," https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3070-3070ti/, 2020.

[50] NVIDIA Corporation, "NVIDIA ampere GA102 GPU architecture whitepaper," https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf, 2021.

[51] L. Nyland, J. R. Nickolls, G. Hirota, and T. Mandal, "Systems and methods for coalescing memory accesses of parallel threads," US Patent No. 8,086,806, 2011.

[52] C. H. Park, T. Heo, J. Jeong, and J. Huh, "Hybrid TLB coalescing: Improving TLB translation coverage under diverse fragmented memory allocations," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2017.

[53] J. Park, Y. L. Kwon, S. Jeong, G. B. Hong, J. Yoon, P. J. Nair, and S. Hong, "A case for speculative address translation with rapid validation for GPUs," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.

[54] B. Pham, A. Bhattacharjee, Y. Eckert, and G. H. Loh, "Increasing TLB reach by exploiting clustering in page translations," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[55] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, "CoLT: Coalesced large-reach TLBs," in *Proc. International Symposium on Microarchitecture (MICRO)*, 2012.

[56] B. Pichai, L. Hsu, and A. Bhattacharjee, "Architectural support for address translation on GPUs: Designing memory management units for CPU/GPUs with unified address spaces," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014.

[57] J. Power, M. D. Hill, and D. A. Wood, "Supporting x86-64 address translation for 100s of GPU lanes," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[58] J. Power, M. D. Hill, and D. A. Wood, "Supporting x86-64 address translation for 100s of GPU lanes," in *Proc. International Symposium on High-Performance Computer Architecture (HPCA)*, 2014.

[59] M. K. Qureshi, D. N. Lynch, O. Mutlu, and Y. N. Patt, "A case for MLP-aware cache replacement," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2006.

[60] V. Seshadri, O. Mutlu, M. A. Kozuch, and T. C. Mowry, "The evicted-address filter: A unified mechanism to address both cache pollution and thrashing," in *Proc. International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2012.

[61] S. Shin, G. Cox, M. Oskin, G. H. Loh, Y. Solihin, A. Bhattacharjee, and A. Basu, "Scheduling page table walks for irregular GPU applications," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2018.

[62] D. Skarlatos, A. Kokolis, T. Xu, and J. Torrellas, "Elastic cuckoo page tables: Rethinking virtual memory translation for parallelism," in *Proc. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[63] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," University of Illinois at Urbana-Champaign, Tech. Rep. IMPACT-12-01, 2012.

[64] Y. Wang, B. Li, M. T. Ibn Ziad, A. Jaleel, J. Yang, and X. Tang, "OASIS: Object-aware page management for multi-GPU systems," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2025.

[65] Z. Yan, D. Nellans, D. Lustig, and A. Bhattacharjee, "Translation ranger: Operating system support for contiguity-aware TLBs," in *Proc. International Symposium on Computer Architecture (ISCA)*, 2024.